



华南师范大学

本科学生实验（实践）报告

院 系： 计算机学院

实验课程： 编译原理

实验项目： 实验2：正则表达式转换为DFA图

指导老师： 黄煜廉

开课时间： 2023 ~ 2024 年度第 2 学期

专 业： 计算机科学与技术

班 级： 22级计算机科学与技术2班

学生姓名： 黄兆清

华南师范大学教务处

华南师范大学实验报告

学生姓名 黄兆清 学 号 20222131044
专 业 计算机科学与技术 年级、班级 22级计科2班
课程名称 编译原理 实验项目 实验2：正则表达式转换为DFA图
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2024 年 4 月 1 日
实验指导老师 黄煜廉 实验评分

一、实验内容

设计一个应用软件，以实现将正则表达式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ DFA 最小化

- (1) 正则表达式应该可以支持命名操作，运算符有：转义符号(\)、连接、选择(|)、闭包(*)、正闭包(+)、[]、可选(?)、括号()
- (2) 要提供一个源程序编辑界面，让用户输入一行(一个)或多行(多个)正则表达式(可保存、打开正则表达式文件)
- (3) 需要提供窗口以使用户可以查看转换得到的 NFA (用状态转换表呈现即可)
- (4) 需要提供窗口以使用户可以查看转换得到的 DFA (用状态转换表呈现即可)
- (5) 需要提供窗口以使用户可以查看转换得到的最小化 DFA (用状态转换表呈现即可)
- (6) 要求应用程序为 Windows 界面
- (7) 书写完善的软件文档

二、实验目的

通过设计和实现一个正则表达式转换工具软件，加深对正则表达式到有限自动机的转换过程的理解，并掌握相关算法的实现原理与方法。

1. 理解正则表达式的基本语法和运算符，并能够正确地将其转换为对应的非确定有限自动机 (NFA)。
2. 掌握将 NFA 转换为确定有限自动机 (DFA) 的算法，了解 DFA 相较于 NFA 的优劣和应用场景。
3. 理解 DFA 的最小化算法，学会如何将 DFA 简化为最小化的等价 DFA，从而减少状态的数量，提高自动机的效率和性能。

4. 学习使用 Windows 界面编程，设计和实现一个用户友好的应用程序，使用户能够输入正则表达式、查看转换后的 NFA、DFA 以及最小化 DFA，并保存/打开正则表达式文件。

三、实验文档：

1. 实验项目概述：

- 本实验旨在设计和实现一个正则表达式转换工具软件，该软件能够将用户输入的正则表达式转换为对应的非确定有限自动机（NFA），进而转换为确定有限自动机（DFA），最后将 DFA 最小化。同时，该软件提供了源程序编辑界面，用户可以输入一行或多行正则表达式，并可以保存和打开正则表达式文件。整个软件采用 Windows 界面。
- 主要功能包括：
 - 1. 正则表达式解析与转换：支持正则表达式的基本语法和运算符，包括转义符号 (\)、连接、选择 (|)、闭包 (*)、正闭包 (+)、[]、可选 (?)、括号 ()，并支持命名操作。
 - 2. NFA 可视化：提供窗口以便用户可以查看转换得到的 NFA，使用状态转换表呈现。
 - 3. DFA 可视化：提供窗口以便用户可以查看转换得到的 DFA，使用状态转换表呈现。
 - 4. 最小化 DFA 可视化：提供窗口以便用户可以查看转换得到的最小化 DFA，使用状态转换表呈现。
 - 5. 源程序编辑界面：提供一个用户友好的源程序编辑界面，允许用户输入、保存和打开正则表达式文件。

2. 实验项目设计

- 需求分析：首先确定了实验项目的需求，即设计一个应用软件，能够实现将正则表达式转换为非确定有限自动机（NFA）、确定有限自动机（DFA）、以及最小化的 DFA。
- 功能设计：确定了软件需要具备的功能，包括：

- 提供源程序编辑界面，让用户输入正则表达式。
 - 支持多行正则表达式输入，并提供保存和打开功能。
 - 提供窗口以使用户查看转换得到的 NFA、DFA、以及最小化的 DFA，使用状态转换表呈现。
 - 实现将正则表达式转换为 NFA、DFA 以及最小化的 DFA 的算法。
- 界面设计：确定了软件的界面设计，要求为 Windows 界面，包括源程序编辑界面和结果展示窗口。
 - 数据结构设计：设计了适合存储正则表达式、NFA、DFA 等数据的数据结构，例如 ``scanData``、``finalData``、``edge`` 和 ``graph``。
 - 算法设计：设计了将正则表达式转换为 NFA、DFA、最小化 DFA 的算法，包括处理正则表达式、构建 NFA 图、NFA 转换为 DFA、以及 DFA 最小化等过程。
 - 模块设计：根据功能和算法的不同，设计了相应的模块，如扫描模块、转换模块、界面模块等。
 - 测试设计：设计了相应的测试用例，用于验证软件功能的正确性和稳定性。

3. 数据结构设计

- `scanData` 结构体：
 - ``bool needToScan``: 一个布尔值，表示是否需要对该正则表达式进行扫描。如果为 `true`，则表示需要进行扫描；如果为 `false`，则表示不需要进行扫描。
 - ``std::string name``: 一个字符串，表示正则表达式的名称或标识符。
 - ``std::string re``: 一个字符串，表示待处理的正则表达式。

这个结构体主要用于存储从用户输入界面获取的原始数据，包括正则表达式的名称和内容，以及是否需要对该正则表达式进行扫描的信息。

- finalData 结构体:

- `std::string name`: 一个字符串, 表示正则表达式的名称或标识符。
- `std::string reExpress`: 一个字符串, 表示转换后的正则表达式。

这个结构体用于存储转换后的结果数据, 包括转换后的正则表达式名称和内容。当用户输入的正则表达式经过转换生成了最终的结果, 例如 NFA、DFA 或最小化的 DFA, 这些结果会被存储在这个结构体中, 以便后续的展示和使用。

- edge 结构体:

- `int begin`: 边的起始节点。
- `int end`: 边的结束节点。
- `bool calEx`: 是否为判定条件。
- `std::string express`: 表达式。

这个结构体表示图中的一条边, 其中包含了边的起始节点、结束节点、是否为判定条件以及表达式内容。该结构体还提供了构造函数和一个用于生成字符串表示的方法 `toString()`。

- graph 结构体:

- `int start`: 图的起始节点。
- `int end`: 图的结束节点。
- `int size`: 图的规模。
- `std::unordered_set<int> finalNodes`: 存储终止节点的集合。
- `std::vector<edge> *edges`: 存储图的所有边的指针。

这个结构体表示一个图, 其中包含了图的起始节点、结束节点、规模以及终止节点的集合。`edges` 成员是一个指向 `edge` 结构体的向量的指针, 用于存储图的所有边。

- `Gragh` 类:

- ``gragh *inGragh``: 指向一个 ``gragh`` 结构体的指针, 表示当前处理的图。
- ``std::stack<edge> subG``: 用于辅助处理的边栈。
- ``std::stack<sign> signs``: 用于辅助处理的符号栈。
- ``std::string reExpress``: 存储正则表达式。

这个类提供了处理图的方法, 包括从正则表达式生成 NFA、将 NFA 转换为 DFA 等。其中涉及的方法有 ``toNFA()``、``toDFA()``、``compressDFA()`` 等。成员函数 ``toString()`` 用于将图转换为字符串表示。

4. 关键算法设计

第一部分: 将带标识符的输入, 转换为可识别的规范的正则表达式

函数 ``scanner`` 的设计思路和算法如下:

- 扫描所有句子 (``scanAll`` 函数):

- 遍历输入的字符串向量 ``lines`` 中的每一行。
- 对于每一行, 首先初始化一个 ``scanData`` 结构体指针 ``oneData``, 表示当前处理的数据。
- 设置一个指示变量 ``startPos``, 用于记录等号前面的位置。
- 使用一个 ``while`` 循环遍历当前行的字符, 处理等号前后的部分。
- 如果字符为空格或制表符, 则跳过。
- 如果 ``startPos`` 为 0, 表示还未记录等号前面的内容, 此时检查当前字符是否为下划线, 如果是则将 ``needToScan`` 置为 `true`, 并将 ``startPos`` 更新为当前位置, 表示开始记录标识符。
- 如果当前字符为等号, 则将等号前的内容作为标识符, 并将等号后的内容作为正则表达式内容, 存储到 ``oneData`` 中, 并将 ``oneData`` 存储到 ``result`` 中。
- 返回存储了所有 ``scanData`` 结构体指针的 ``result``。

- 替换扫描结果中的正则表达式 (``replaceRe`` 函数):

- 创建一个 ``std::map<std::string, std::string>``, 用于存储标识符与对应正则表达式

的映射关系。

- 遍历所有扫描到的正则表达式，并将不需要扫描的正则表达式的标识符和对应的正则表达式存储到映射中。

- 对于每个需要扫描的正则表达式，遍历其正则表达式内容中的字符，查找是否存在其他已定义的标识符，如果有则将其替换为对应的正则表达式定义。

- 将替换后的正则表达式存储到 `finalData` 结构体指针中，并返回存储了所有 `finalData` 结构体指针的 `temp`。

- 基础化替换 (`baseRe` 函数):

- 对于每个 `finalData` 结构体指针中的正则表达式内容，遍历其中的字符。

- 如果遇到 `[` 符号，表示需要进行替换。

- 在 `[` 符号后搜索 `]` 符号，找到闭合的区间。

- 对于闭合区间内的字符，根据连字符 `-` 进行扩展，生成对应的字符，并将其替换为括号内的形式。

- 返回处理后的 `finalData` 结构体指针。

- 主函数 `scanner`:

- 首先调用 `scanAll` 函数扫描所有句子，得到扫描结果。

- 将扫描结果传递给 `replaceRe` 函数进行替换，得到替换后的结果。

- 将替换后的结果传递给 `baseRe` 函数进行基础化替换，得到最终的结果。

- 返回最终的结果。

第二部分：将规范的正则表达式，转换为 NFA 图

- toNFA 函数:

- 遍历输入的正则表达式 `re` 中的每个字符。

- 根据当前字符的类型（符号或者普通字符），判断是否需要进行转义。

- 如果当前字符是正则表达式的运算符，则根据优先级和栈顶元素的运算符类型，决定是否需要入栈、出栈或者添加边等操作。

- 如果当前字符是普通字符，则创建新的边并入栈。

- 最终，根据栈中剩余的操作符，生成对应的 NFA 图。

- 其他辅助函数：

- `aNewEdge`：用于创建一个新的边，并将其入栈。

- `andConnet`：用于连接两个 NFA 图的结束节点和开始节点。

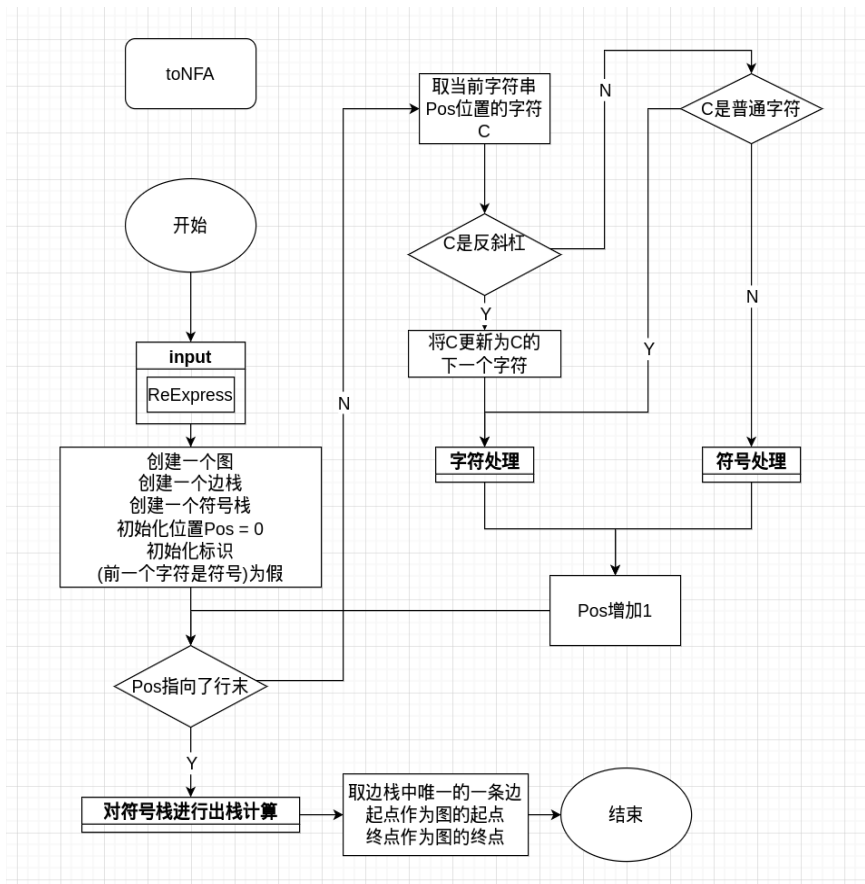
- `orConnet`：用于创建两个新的节点，并将其与已有的节点进行连接，实现逻辑或操作。

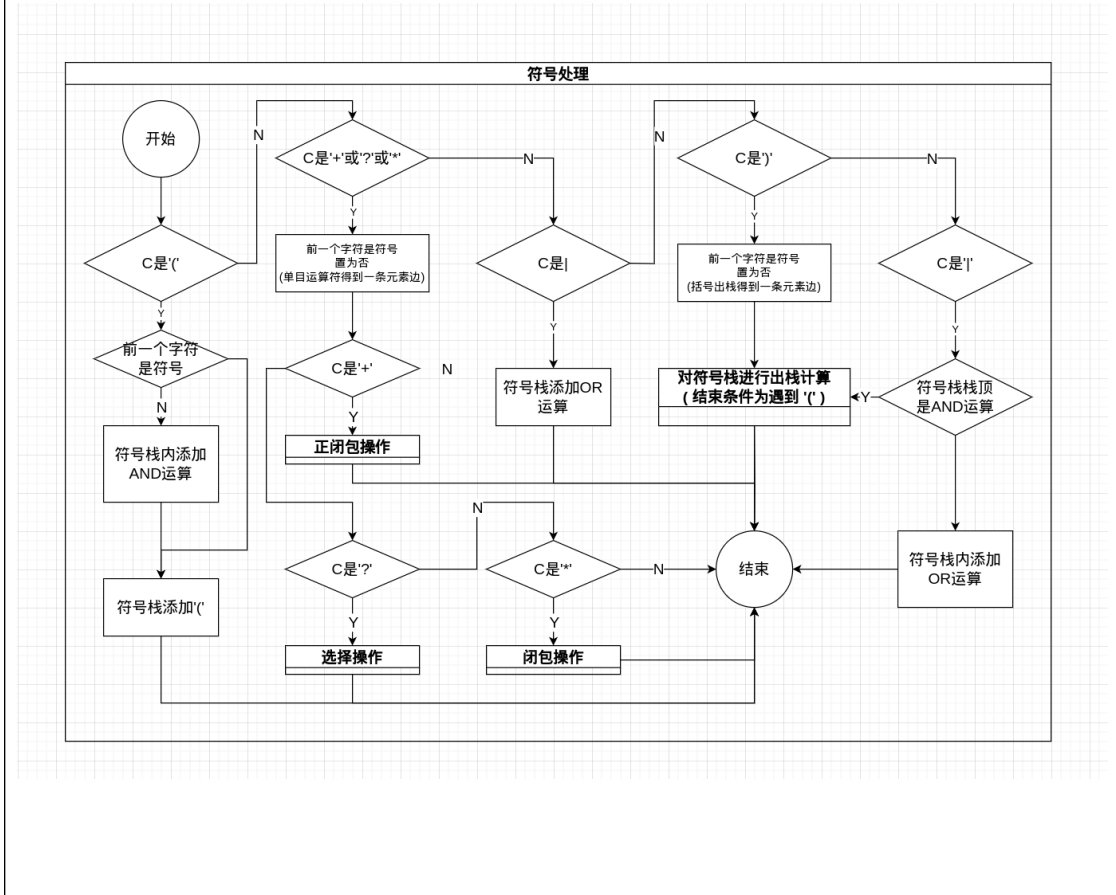
- `closure`：用于对一个节点进行闭包操作，即在其前后添加空边，并创建新的开始和结束节点。

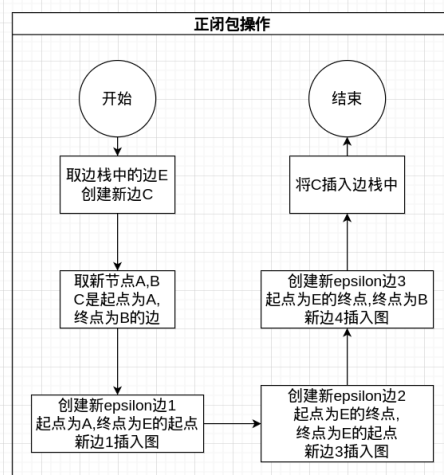
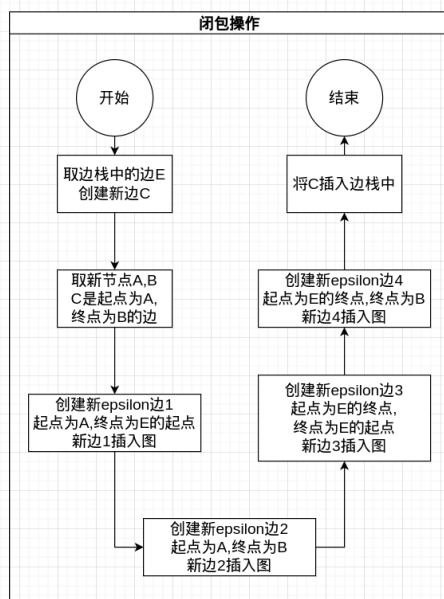
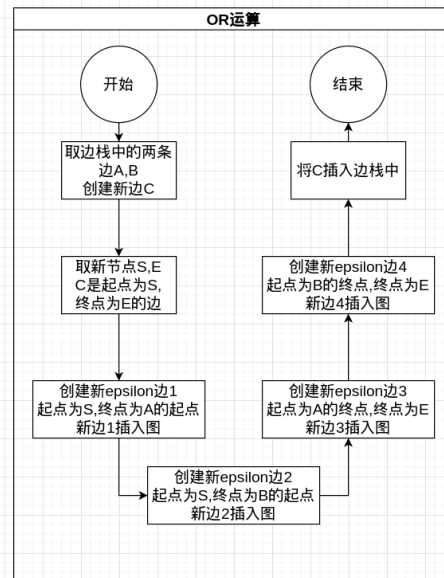
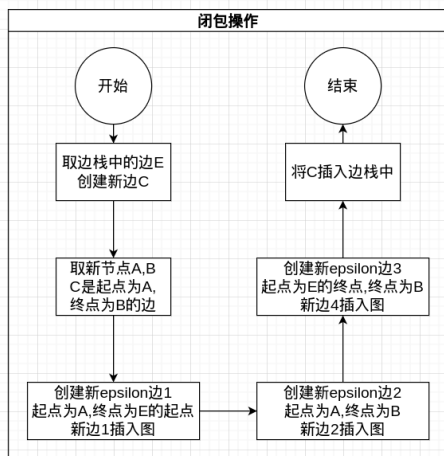
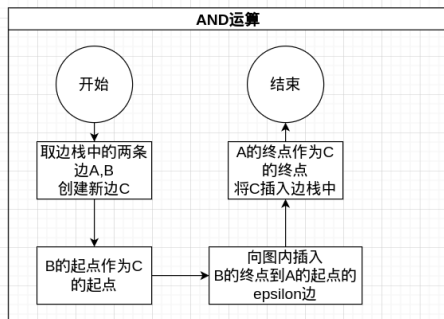
- `qmConnet`：用于对一个节点进行零次或一次的操作，类似于可选操作。

- `positive\_closure`：用于对一个节点进行正闭包操作，即在其前后添加空边，并创建新的开始和结束节点。

- 流程图如下：





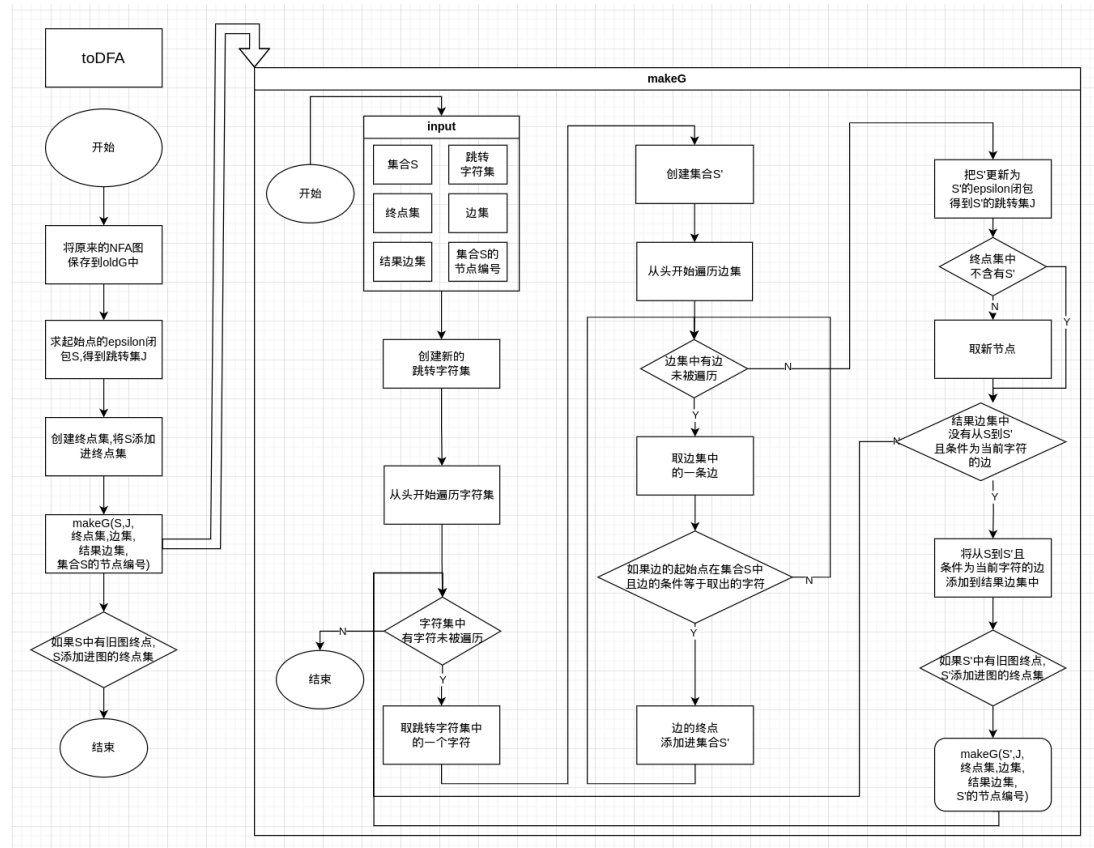


第三部分：将规范 NFA，转换为 DFA 图

- `eclosure` 函数：
 - 从给定的起始节点集合 ``starts`` 出发，寻找所有可以通过空边直接到达的节点，并返回这些节点对应的字符集合。
 - 遍历起始节点集合，查找所有以起始节点为起点且标记为可为空边的边。
 - 如果找到空边，则将空边的终点节点加入到结果集合中，并继续递归地查找这些终点节点的 e 闭包。
 - 最终返回所有 e 闭包节点对应的字符集合。
- `strJump` 函数：
 - 给定一个状态集合 ``start`` 和字符 ``str``，返回所有通过字符 ``str`` 可以到达的节点集合。
 - 遍历起始状态集合，查找所有以起始节点为起点且标记为字符 ``str`` 的边。
 - 将这些边的终点节点加入到结果集合中，并返回。
- `search` 函数：
 - 在状态集合的向量 ``nodeVec`` 中查找是否存在与给定状态集合 ``state`` 相同的状态集合，并返回其索引，若不存在则返回 -1。
- `makeG` 函数：
 - 根据 NFA 图 ``G`` 的节点集合和字符集合，构建 DFA 图，并返回结果集合。
 - 遍历字符集合 ``jumps``，对每个字符，根据当前节点集合 ``start`` 和字符，通过 ``strJump`` 函数获取可以到达的下一状态集合，并计算其 e 闭包。
 - 判断下一状态集合是否已经存在于节点向量 ``nodeVec`` 中，若不存在则加入，并递归地调用 ``makeG`` 函数继续构建 DFA 图。
 - 将生成的边添加到结果集合中，并标记终点节点是否为终止状态。
- `toDFA` 函数：
 - 根据当前的 NFA 图，构建对应的 DFA 图。

- 初始化起始状态集合为 NFA 图的起始节点, 通过 `eclosure` 函数获取其 e 闭包。
- 判断起始状态是否为终止状态, 并标记为终止状态。
- 调用 `makeG` 函数构建 DFA 图, 并将结果添加到新的 DFA 图中。

- 流程图如下:



第四部分：将 DFA 图最小化

这些函数实现了从 DFA 图到最小化 DFA 图的转换逻辑。以下是转换的简述：

- compressDFA 函数：
 - 首先创建四个闭包集合 `closures11`、`closures00`、`closures01`、`closures10`，分别用于记录终止状态到终止状态、非终止状态到非终止状态、非终止状态到终止状态和终止状态到非终止状态的跳转。
 - 遍历 DFA 图的边, 根据边的起点和终点的终止状态属性, 将节点分别添加到对

应的闭包集合中。

- 构建节点属性二进制串，根据节点所在闭包集合的情况，将终止状态和非终止状态分别用 1 和 0 表示，并根据闭包集合中是否包含当前节点确定该节点在二进制串中的位置。

- 根据节点属性对节点进行排序，并根据属性相同的节点进行合并，得到节点合并集合 `unionVec`。

- 创建新的 DFA 图，并根据节点合并集合构建图的节点和边，将原 DFA 图中的边映射到新图中。

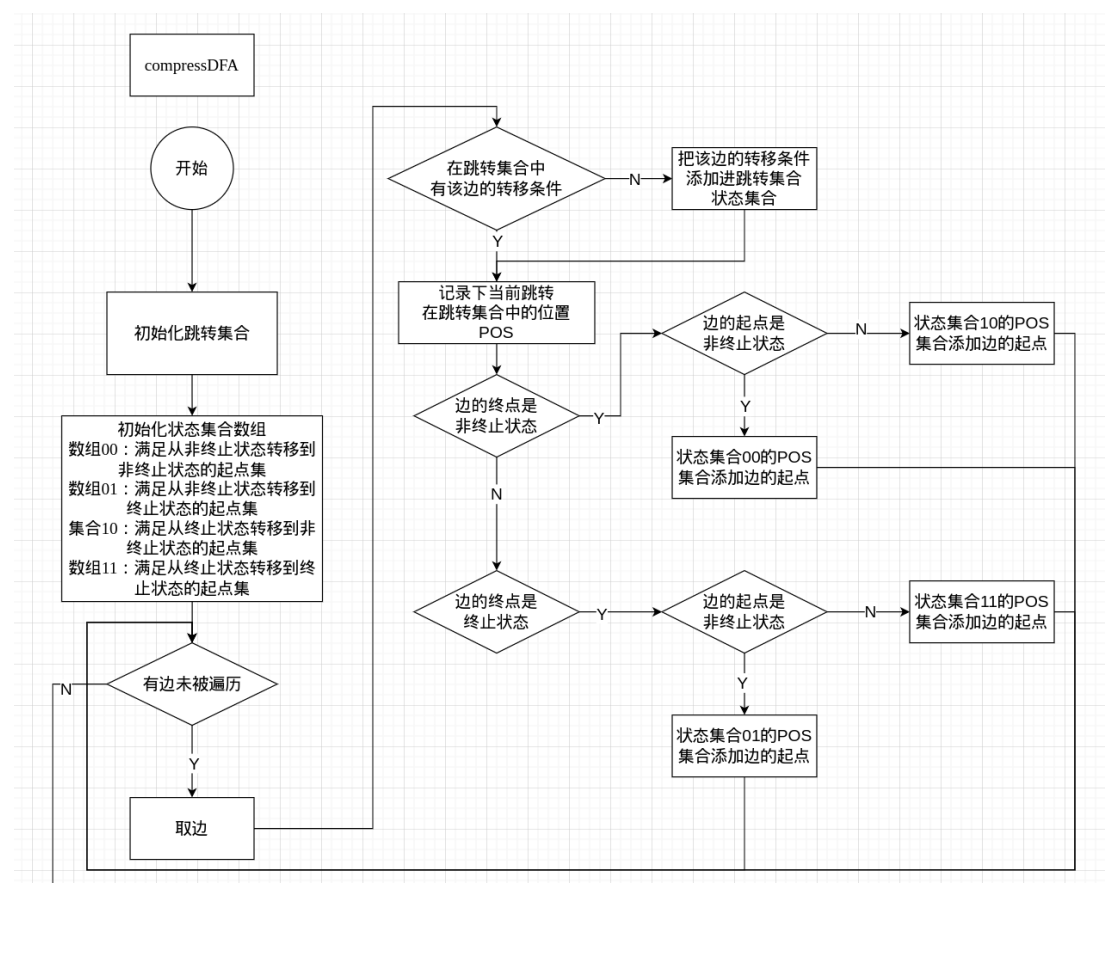
- 将原 DFA 图的起始状态和终止状态对应到新图中，并将边添加到新图中。

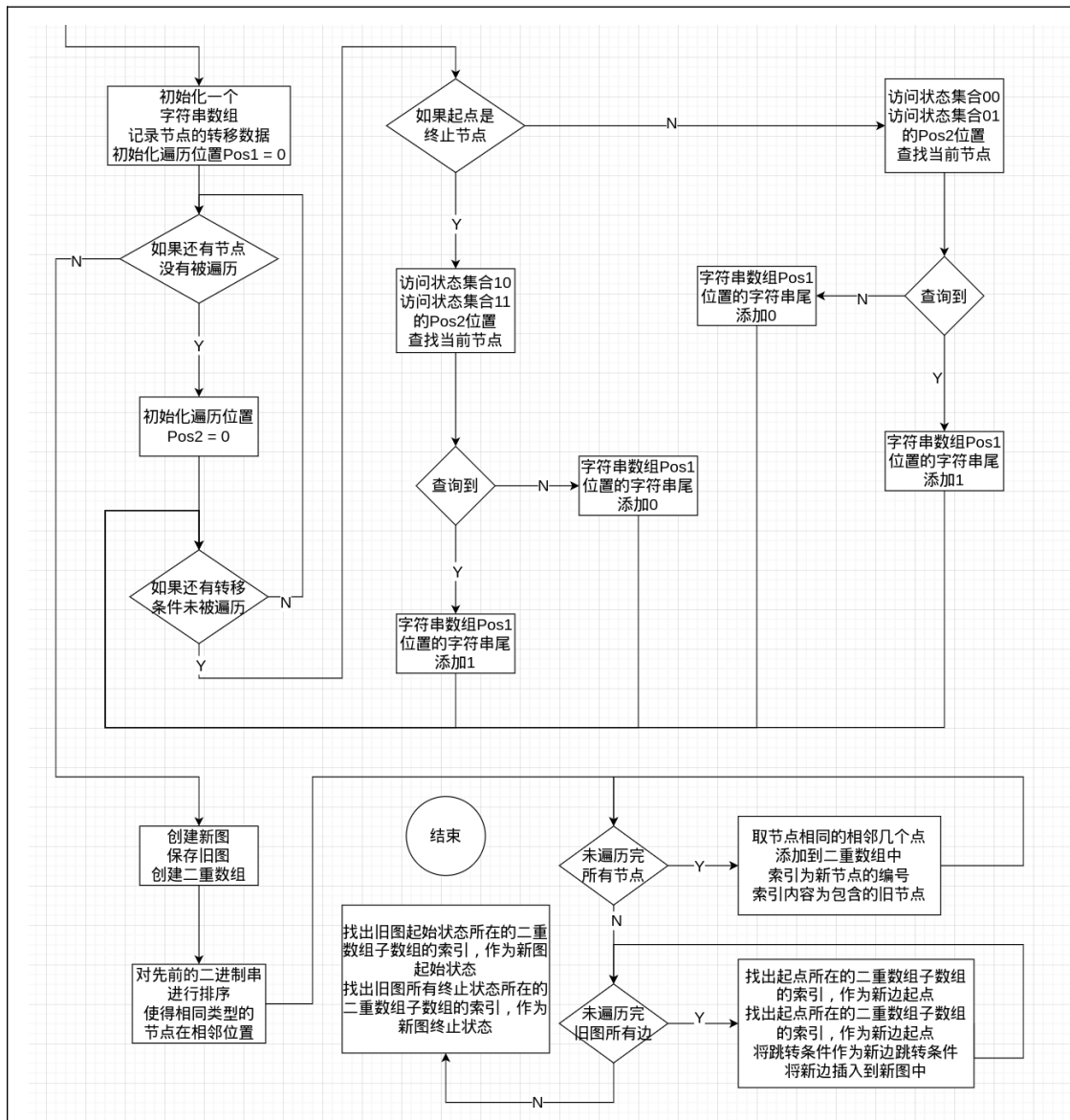
- 最后删除原 DFA 图，并将新图设置为当前 DFA 图。

- cmpStr 函数：

- 用于在排序节点属性时作为比较函数，按照节点属性字符串进行升序排序。

- 流程图如下：





• 测试数据组成

◆ 所有基础运算检查：

_test0 = a

_test1 = ab

_test2 = a|b

_test3 = a*

_test4 = a+

_test5 = a?

◆ **组合基础运算检查：**

_test6 = ab|c

_test7 = (a|b)c

_test8 = c(a|b)

_test9 = b(a*)

_test10 = a\+?

_test11 = a*(b)

_test12 = [1-5]

_test13 = [1-2]c

_test14 = [1-2]|c

◆ **复杂运算检查：**

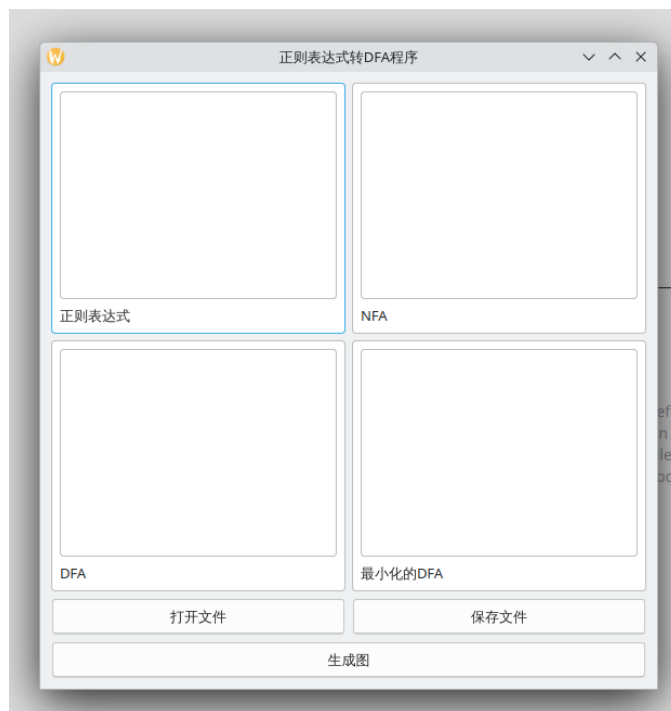
digit = [0-9]

letter = [a-zA-Z]

_ID = letter(letter|digit)*

_Exp = digit+(\+|-)digit+

• **图形界面样式：**



四、实验总结（心得体会）

在这个实验中，我学到了许多关于有限状态自动机（DFA）以及正则表达式相关的知识，并且通过编写代码实现了从正则表达式到最小化 DFA 图的转换过程。以下是我的实验总结和心得体会：

1. 深入理解 DFA：通过这个实验，我对有限状态自动机（DFA）的工作原理有了更深入的理解。我学会了如何将正则表达式转换为 NFA 图，然后再将 NFA 图转换为 DFA 图，最后将 DFA 图最小化，以便更高效地进行字符串匹配。
2. 理解正则表达式：实验让我更加深入地理解了正则表达式的语法和运算规则，包括闭包、正闭包、或运算等。通过编写代码实现了这些操作，加深了对正则表达式的理解。
3. 算法实现与优化：在实现代码的过程中，我遇到了许多算法设计和优化的问题。特别是在将 DFA 图最小化的过程中，需要考虑如何合并状态以及如何优化遍历和搜索过程，这让我对算法的设计和优化有了更深入思考。
4. 实践能力提升：通过编写代码实现算法，我对课程所学知识有了更深刻的理解，并且提升了自己的实践能力。在解决问题的过程中，我学会了如何分析问题、寻找解决方案，这对我的职业发展和学术研究都是非常有价值的。

同时这次实验的完成还有一些不足之处：

1. 图转换算法效率不高：在实验中，图的转换算法可能存在一些效率问题，特别是在将正则表达式转换为 NFA 图和将 NFA 图转换为 DFA 图的过程中。这些算法的时间复杂度可能较高，导致程序的执行速度较慢。
2. 数据结构选择不当：在实现图的转换算法时，可能选择了不够高效的数据结构，导致了一些操作的时间复杂度较高。例如，可能使用了线性搜索而不是更高效的数据结构来查找状态之间的转换关系，或者使用了不够紧凑的数据结构来表示图。

3. 算法优化不足：在实验中可能存在一些算法优化的空间没有被充分利用。例如，在将 DFA 图最小化的过程中，可能存在一些可以优化的地方，如减少状态的比较次数、提前终止搜索等，从而降低算法的时间复杂度。

4. 缺乏并行化和优化策略：程序可能缺乏并行化和优化策略，无法充分利用现代计算机的多核处理器和高级编译器的优化功能，导致程序的执行效率较低。

五、参考文献：

[1] Kenneth C.Louden.编译原理及实践[M].机械工业出版社,2000-3-1.

[2] 陆文周.Qt5 开发及实例（第 2 版）[M].第 2 版.电子工业出版社,2015-5-1.

--